C++ for Interviews

Code-Trace MCQ Question Set

After Dark · Codeforces: After_Dark

Chapters 1–26 · 3–4 questions per section · answers with explanations at the end

(S) = single correct answer **(M)** = one or more correct

Contents

Part 1 Foundations

Ch. 1 Preprocessor, Headers & Compilation

Q1 S — Compilation stages §1.1, p9

Which is the correct order of the three C++ build stages?

(A) Compiler → Preprocessor → Linker

(B) Preprocessor → Linker → Compiler

(C) Preprocessor → Compiler → Linker

(D) Linker → Compiler → Preprocessor

Q2 S — Template definition placement §1.5, p11

Why must template definitions go in header files rather than `.cpp` files?

(A) Templates are a linker feature and the linker needs them.

(B) The compiler needs the full template body at every instantiation site in every translation unit.

(C) `#pragma once` only works in headers.

(D) Templates are macro-expanded by the preprocessor, not the compiler.

Q3 M — #pragma once vs include guards §1.2, p9

Which statements are correct? (Pick ONE or MORE)

☐ **A)** `#pragma once` lets the compiler skip re-reading the file entirely.

☐ **B)** Traditional include guards use `#ifndef`/`#define`/`#endif`.

☐ **C)** `#pragma once` is part of the C++ standard.

☐ **D)** Both techniques prevent multiple-inclusion redefinition errors.

Q4 S — Forward declaration: what is allowed? §1.3, p9

Given only `class Foo;`, which of the following compiles?

(A) `Foo obj;` (object by value)

(B) `Foo* ptr = nullptr;` (pointer to Foo)

(C) `ptr->doSomething();` (calling a method via pointer)

(D) `std::cout << sizeof(Foo);`

Ch. 2 Types, Namespaces & const/constexpr

Q5 S — Output of scope resolution §2.2.2, p14

```
int x = 100;
void demo() {
    int x = 5;
    std::cout << x << " " << ::x << "\n";
}
```

What is printed?

(A) `100 5`

(B) `5 5`

(C) `5 100`
(D) `100 100`

Q6 S — constexpr function: when does it run? §2.3.3–6, p15

```
constexpr int square(int x) { return x * x; }
int n; std::cin >> n;
constexpr int a = square(5);   // (1)
const int b     = square(n);   // (2)
auto c          = square(3);   // (3)
```

Which lines trigger compile-time evaluation?
(A) Only (1)
(B) (1) and (2)
(C) (1) and (3)
(D) All three

Q7 S — Useless const on return type §2.3.6, p18

```
const int f() { return 42; }
constexpr auto x = f();   // line A
const auto y     = f();   // line B
auto z           = f();   // line C
```

Which line(s) cause a compile error?
(A) Line A only
(B) Line B only
(C) Lines A and C
(D) No errors — all three compile

Q8 M — `const` member qualifier vs `constexpr` specifier §2.3.4, p16

Which statements are correct? (Pick ONE or MORE)
☐ **A)** `const` on a member function prevents it from modifying `*this`.
☐ **B)** A `constexpr` member function can modify the object (C++14+) if not `const`.
☐ **C)** A function can be both `constexpr` and `const`.
☐ **D)** `constexpr` implies `inline`, so it is safe to define in a header.

Ch. 3 Literals, Type Aliases & auto

Q9 S — auto type deduction §3.3, p22

```
const std::string& get_name();
auto   name1 = get_name();   // (1)
const auto& name2 = get_name();  // (2)
```

What are the types of `name1` and `name2`?
(A) `name1: const string&;  name2: const string&`
(B) `name1: string` (copy); `name2: const string&`
(C) `name1: string&;  name2: string` (copy)
(D) Both are `string` copies

Q10 S — Integer literal suffix §3.1.1, p21

```
auto a = 42;     // (1)
auto b = 42u;    // (2)
auto c = 42LL;   // (3)
auto d = 42.0;   // (4)
```

What are the deduced types of `a`, `b`, `c`, `d`?

(A) `int`, `unsigned`, `long long`, `double`

(B) `int`, `int`, `long`, `float`

(C) `long`, `unsigned long`, `long long`, `double`

(D) `int`, `unsigned int`, `long`, `float`

Q11 S — `decltype` result §3.3.1, p23

```
int x = 5;
decltype(x)   y = 10;
decltype(x + 3.0) z = 0.0;
```

What are the types of `y` and `z`?

(A) `int`, `int`

(B) `int`, `double`

(C) `double`, `double`

(D) `int`, `float`

Ch. 4 Operators In Depth

Q12 S — Integer division and modulo §4.1, p25

What are the values of `a`, `b`, `c`?

```
int a = -7 / 2;
int b = -7 % 2;
int c =  7 % -2;
```

(A) a=−4, b=1, c=−1

(B) a=−3, b=−1, c=1

(C) a=−3, b=1, c=−1

(D) a=−4, b=−1, c=1

Q13 S — Pre vs post increment §4.2, p25

```
int a = 5;
int b = ++a;  // (1)
int c = a++;  // (2)
// What are a, b, c after both lines?
```

(A) a=7, b=6, c=7

(B) a=7, b=6, c=6

(C) a=6, b=6, c=5

(D) a=7, b=7, c=6

Q14 S — Bitwise operations §4.3, p25

```
unsigned a = 0b1010;  // 10
unsigned b = 0b1100;  // 12
std::cout << (a & b) << " " << (a | b) << " " << (a ^ b);
```

(A) 8 14 6
(B) 12 14 2
(C) 8 12 4
(D) 10 14 6

Q15 S — sizeof on pointer vs array §4.6, p27

```
int arr[10];
void broken(int* arr) {
    std::cout << sizeof(arr);   // inside function
}
broken(arr);
std::cout << sizeof(arr);       // outside function (global scope)
```

On a 64-bit system, what are the two outputs?

(A) 40 then 40
(B) 8 then 40
(C) 40 then 8
(D) 8 then 8

Ch. 5 Control Flow

Q16 S — Switch fallthrough output §5.2, p29

```
int x = 2;
switch (x) {
  case 1: std::cout << "one ";
  case 2: std::cout << "two ";
  case 3: std::cout << "three ";
  default: std::cout << "other";
}
```

What is printed?

(A) two
(B) two three
(C) two three other
(D) Compile error

Q17 S — Variable init in switch case §5.2, p30

```
int choice = 2;
switch (choice) {
  case 1:
    int x = 5;    // line A
    break;
  case 2:
    std::cout << x;  // line B
    break;
}
```

(A) Compiles fine; prints `5`
(B) Compiles fine; prints `0`
(C) Compile error: jump over initialization of `x`
(D) Runtime undefined behaviour

Q18 S — Range-based for: copy vs reference §5.3.4, p31

```
std::vector<int> v = {1, 2, 3};
for (int x : v) x *= 2;          // (A)
for (int& x : v) x *= 2;         // (B)
// After both loops, v = ?
```

(A) `{4, 8, 12}` — both loops modify the vector
(B) `{2, 4, 6}` — only loop B modifies the vector
(C) `{1, 2, 3}` — neither loop modifies the vector
(D) `{2, 4, 6}` — only loop A modifies the vector

Ch. 6 Functions In Depth

Q19 S — Default arguments: valid or not? §6.1, p33

Which declaration is **invalid**?

(A) `void f(int a, int b = 2, int c = 3);`
(B) `void f(int a = 1, int b, int c = 3);`
(C) `void f(int a, int b, int c = 3);`
(D) `void f(int a = 1, int b = 2, int c = 3);`

Q20 S — swap without reference §6.1, 10.1, p33

```
void swap(int a, int b) {
    int tmp = a;
    a = b;
    b = tmp;
}
int x = 10, y = 20;
swap(x, y);
std::cout << x << " " << y;
```

(A) `20 10` — swap succeeded
(B) `10 20` — swap had no effect (pass by value)
(C) Compile error: `swap` is already defined in `<algorithm>`
(D) Undefined behaviour

Q21 S — Overload resolution: return type §6.2, p33

```
int  get() { return 1; }
double get() { return 1.0; }  // line X
```

What happens at line X?

(A) Legal — overloads differing in return type are valid

(B) Compile error: redefinition of `get`

(C) Legal — the compiler uses the return type to disambiguate

(D) Legal only if they differ in `noexcept`

Q22 S — Inline functions and headers §6.3, p33

`inline` on a function in a header primarily means:

(A) The compiler is forced to inline the call at every call site.

(B) Multiple identical definitions across translation units are allowed (no ODR violation).

(C) The function runs at compile time.

(D) The function is hidden from the linker.

Ch. 7 I/O, Streams & Fast I/O

Q23 S — FastIO impact §7.1.2, p37

After calling `std::ios::sync_with_stdio(false); std::cin.tie(nullptr);`, which becomes unsafe?

(A) Using `cin` and `cout` in the same program

(B) Mixing `printf`/`scanf` with `cin`/`cout`

(C) Using `cerr` for error output

(D) Calling `cout.flush()` manually

Q24 M — Stream state bits §7.1.5, p38

Which are correct? (Pick ONE or MORE)

☐ **A)** `failbit` is set when a type-conversion error occurs (e.g., reading letters into an `int`).

☐ **B)** `eofbit` is set when the end of the stream is reached.

☐ **C)** After setting `failbit`, subsequent reads will silently succeed.

☐ **D)** Calling `cin.clear()` resets all error bits.

Q25 S — `cerr` vs `clog` §7.1.4, p38

(A) Both `cerr` and `clog` are unbuffered.

(B) `cerr` is unbuffered (flushes every write); `clog` is buffered.

(C) `cerr` is buffered; `clog` is unbuffered.

(D) Both are buffered and flush only on `endl`.

Ch. 8 Strings, string_view & Ranges

Q26 S — string_view dangling reference §8.2, p46

```
std::string_view sv;
{
    std::string s = "hello";
    sv = s;
}
std::cout << sv;  // line X
```

What happens at line X?

(A) Prints `hello` safely (string_view copies the data)

(B) Undefined behaviour — `sv` is a dangling view

(C) Compile error: cannot assign `string` to `string_view`

(D) Prints an empty string

Q27 S — auto type with string literal §8.1.3, p46

```
using namespace std::string_literals;
auto a = "hello";    // (1)
auto b = "hello"s;   // (2)
auto c = "hello"sv;  // (3) -- using std::string_view_literals
```

What are the types of `a`, `b`, `c`?

(A) `string`, `string`, `string_view`

(B) `const char*`, `string`, `string_view`

(C) `const char*`, `const char*`, `string`

(D) `string_view`, `string`, `string_view`

Ch. 9 Scope, Storage Duration & Lifetime

Q28 S — Static local variable §9.2, p49

```
int counter() {
    static int n = 0;
    return ++n;
}
std::cout << counter() << " " << counter() << " " << counter();
```

(A) `0 0 0`

(B) `1 1 1`

(C) `1 2 3`

(D) `3 2 1`

Q29 M — Static variable properties §9.2–9.3, p49

Which are correct? (Pick ONE or MORE)

☐ **A)** A `static` local variable is initialized only once (the first time that scope is entered).

☐ **B)** `static` local variables are stored in the heap.

☐ **C)** In C++11+, initialization of `static` local variables is thread-safe.

☐ **D)** `thread_local` gives each thread its own copy of the variable.

Ch. 10 References

Q30 S — Reference vs pointer: swap §10.1, p51

```
void swap(int& a, int& b) {  // note: & (reference)
    int tmp = a; a = b; b = tmp;
}
int x = 10, y = 20;
swap(x, y);
std::cout << x << " " << y;
```

(A) `10 20` — no effect
(B) `20 10` — swap succeeds
(C) Compile error
(D) Undefined behaviour

Q31 S — Const reference extending lifetime §10.1.1, p51

```
const int& ref = 5 + 3;
std::cout << ref;
```

(A) Compile error: cannot bind a const ref to a temporary
(B) UB: the temporary is immediately destroyed
(C) Prints `8` — const reference extends the temporary's lifetime
(D) Prints `0` — the reference is zero-initialized

Part 2 Memory & OOP

Ch. 11 Memory, Pointers & Smart Pointers

Q32 S — sizeof pointer vs value §11.1, p53

On a 64-bit system, what does each line print?

```
double  d   = 3.14;
double* ptr = &d;
std::cout << sizeof(d)   << " ";  // (A)
std::cout << sizeof(ptr) << " ";  // (B)
std::cout << sizeof(*ptr);        // (C)
```

(A) 8 8 8
(B) 8 4 8
(C) 4 8 4
(D) 8 8 4

Q33 S — nullptr dereference §11.1.3, p53

```
int* p = nullptr;
std::cout << *p;
```

(A) Prints 0
(B) Prints some garbage value
(C) Undefined behaviour (likely a segfault at runtime)
(D) Compile error

Q34 S — Global `const` linkage §11.3.1, p55

```
// file_a.cpp
const int LIMIT = 100;

// file_b.cpp
extern const int LIMIT;  // does this work?
```

(A) Works fine — `const` globals have external linkage by default
(B) Linker error — `const` globals default to internal linkage
(C) Works because `inline const` is implied
(D) Compile error in `file_b.cpp`

Q35 M — Implicit conversion ranking §11.4, p58

```
void f(int x) {}
void f(char x) = delete;  // deleted overload

f('a');    // (1)
f(5LL);    // (2)
f(3.14);   // (3)
```

Which calls cause a compile error? (Pick ONE or MORE)

☐ **A)** (1) — exact match on a deleted overload

☐ **B)** (2) — ambiguous: `long long` can convert to both `int` and `char`
☐ **C)** (3) — `double` to `int` is a narrowing conversion
☐ **D)** None — all three compile and call `f(int)`

Ch. 12 Type Conversion & Type Safety

Q36 S — static_cast vs C-style cast §12.2, p59

```
double result = static_cast<double>(7) / 2;  // (A)
double wrong  = (double)(7 / 2);              // (B)
```

What are the values of `result` and `wrong`?

(A) `result = 3.5, wrong = 3.5`
(B) `result = 3.5, wrong = 3.0`
(C) `result = 3.0, wrong = 3.5`
(D) Both are `3.0`

Q37 S — std::optional usage §12.3, p60

```
std::optional<int> find(int key) {
    if (key == 42) return 100;
    return std::nullopt;
}
auto v = find(7);
std::cout << v.value_or(-1);
```

(A) `100`
(B) `0`
(C) `-1`
(D) Throws `std::bad_optional_access`

Ch. 13 Classes & OOP

Q38 S — Scoped enum vs unscoped enum §13.1.1, p61

```
enum Color  { Red, Green };      // unscoped
enum class Dir { Left, Right };  // scoped

int x = Red;          // (A)
int y = Dir::Left;    // (B)
```

(A) Both (A) and (B) compile
(B) (A) compiles; (B) is a compile error (no implicit conversion)
(C) (B) compiles; (A) is a compile error
(D) Neither compiles

Q39 S — Copy constructor: shallow vs deep copy §13.1.4, p62

```
struct Bad {
    int* data;
    Bad(int v) : data(new int(v)) {}
    // No copy constructor defined
};
Bad a(5);
Bad b = a;      // compiler-generated copy
*b.data = 99;
std::cout << *a.data;  // what prints?
```

(A) `5` — deep copy was made
(B) `99` — both point to the same heap int (shallow copy)
(C) Compile error: no copy constructor
(D) UB because `data` is uninitialized

Q40 S — The const member function §13.1.11, p76

```
class Wallet {
    int m_balance = 100;
public:
    int get() const { return m_balance; }
    void deposit(int n) { m_balance += n; }
};
const Wallet w;
w.deposit(50);  // (A)
w.get();        // (B)
```

(A) Both (A) and (B) compile
(B) (A) is a compile error; (B) compiles
(C) (B) is a compile error; (A) compiles
(D) Neither compiles because `w` is `const`

Q41 S — RVO: how many copies? §13.1.5, p68

```
std::string make() {
    return std::string("hello");  // RVO candidate
}
std::string s = make();
```

With mandatory RVO (C++17), how many `std::string` copies are made?
(A) 2 (one inside `make`, one into `s`)
(B) 1 (one copy into `s`)
(C) 0 (the object is constructed directly in `s`'s storage)
(D) Depends on the compiler

Ch. 14 Operators, Containers & Lambdas

Q42 S — emplace_back vs push_back §14.2.1, p86

```
std::vector<std::pair<int,int>> v;
v.push_back({1, 2});    // (A)
v.emplace_back(3, 4);   // (B)
```

Which is more efficient and why?

(A) (A), because `push_back` avoids a heap allocation

(B) (B), because `emplace_back` constructs the object in-place without creating a temporary

(C) Both are identical in performance

(D) (A), because pairs cannot be emplaced

Q43 S — Lambda capture: copy vs reference §14.4.4–6, p93

```
int x = 10;
auto f = [x]() mutable { x = 99; return x; };
auto g = [&x]()        { x = 99; return x; };
f();
std::cout << x << " ";
g();
std::cout << x;
```

(A) `99 99`

(B) `10 99`

(C) `99 10`

(D) `10 10`

Q44 S — Lambda default captures §14.4.4, p93

```
int a = 1, b = 2;
auto lam = [=]() { return a + b; };
a = 100; b = 200;
std::cout << lam();
```

(A) `300` — lambda sees current values of `a`, `b`

(B) `3` — lambda captured copies of `a` and `b` at creation time

(C) `102` — only `b` was captured by value

(D) Compile error

Q45 S — Erase from vector inside loop §14.2.4, p88

```
std::vector<int> v = {1, 2, 3, 4, 5};
for (auto it = v.begin(); it != v.end(); ++it) {
    if (*it % 2 == 0) v.erase(it);  // erase even numbers
}
```

(A) Correctly removes `{2, 4}`; result is `{1,3,5}`

(B) Undefined behaviour: iterator `it` is invalidated after `erase`

(C) Compile error: cannot call `erase` inside range-based loop

(D) Removes all elements

Q46 M — Object relations §14.5, p101

Match each to the correct category: (Pick ONE or MORE that are TRUE)

☐ **A)** Composition ("has-a with ownership"): the owned object's lifetime is tied to the owner.

☐ **B)** Aggregation ("has-a without ownership"): the owned object can outlive the container.

☐ **C)** Association ("uses-a"): no storage of the other object; just interacts via parameters.

☐ **D)** Inheritance is the loosest form of coupling among these four.

Ch. 15 Advanced OOP: Virtual Dispatch

Q47 S — Without virtual: static binding §15.1.2, p108

```
struct Animal {
    void speak() { std::cout << "Animal\n"; }
};
struct Dog : Animal {
    void speak() { std::cout << "Dog\n"; }
};
Animal* p = new Dog();
p->speak();
```

(A) Prints `Dog` — runtime dispatch chooses Dog
(B) Prints `Animal` — static binding based on pointer type
(C) Undefined behaviour
(D) Compile error: Dog shadows Animal::speak

Q48 S — With virtual: runtime dispatch §15.1.2, p108

```
struct Animal {
    virtual void speak() { std::cout << "Animal\n"; }
};
struct Dog : Animal {
    void speak() override { std::cout << "Dog\n"; }
};
Animal* p = new Dog();
p->speak();
```

(A) Prints `Animal`
(B) Prints `Dog`
(C) Prints `Animal\nDog`
(D) Compile error

Q49 S — override catches typo §15.1.7–8, p110

```
struct Base {
    virtual void process(int x) {}
};
struct Derived : Base {
    void process(double x) override {}  // note: double, not int
};
```

(A) Compiles — `override` just marks the intent
(B) Compile error: `override` finds no matching virtual in the base
(C) Runtime error: vtable mismatch
(D) Creates a new overload, hiding the base version

Q50 S — Missing virtual destructor §15.1.5, p109

```
struct Base {
    ~Base() { std::cout << "~Base\n"; }  // NOT virtual
};
struct Derived : Base {
    ~Derived() { std::cout << "~Derived\n"; }
};
Base* p = new Derived();
delete p;
```

(A) Prints `~Derived` then `~Base`
(B) Prints `~Base` only — undefined behaviour (resource leak for Derived members)
(C) Compile error
(D) Prints `~Base` then `~Derived`

Q51 S — Abstract class: pure virtual §15.1.20, p117

```
struct Shape {
    virtual double area() const = 0;  // pure virtual
};
Shape s;           // (A)
Shape* p = new Shape();  // (B)
```

(A) Both (A) and (B) are compile errors
(B) (A) is a compile error; (B) compiles (heap allocation works)
(C) Both compile
(D) (B) compiles; (A) is a compile error

Q52 M — Diamond problem with virtual inheritance §15.1.21–3, p119

```
struct A { int x = 1; };
struct B : A {};
struct C : A {};
struct D : B, C {};
D d;
// d.x = 5;  // what happens here?
```

Without virtual inheritance, which are correct? (Pick ONE or MORE)

☐ **A)** `D` has two copies of `A`'s data members.
☐ **B)** `d.x` is ambiguous — compile error.
☐ **C)** The fix is: `struct B : virtual A {};` and `struct C : virtual A {};`.
☐ **D)** Virtual inheritance adds a pointer (vbptr) to each derived object.

Ch. 16 pair, tuple & variant

Q53 S — std::variant access §16.3, p132

```
std::variant<int, double, std::string> v = 3.14;
std::cout << std::get<int>(v);  // line X
```

(A) Prints `3` (truncation)
(B) Throws `std::bad_variant_access` at runtime
(C) Compile error

(D) Prints `3.14`

Q54 S — Structured bindings with pair §26.2, p169

```
std::map<std::string,int> m = {{"a",1},{"b",2}};
for (const auto& [key, val] : m)
    std::cout << key << val << " ";
```

(A) `a1 b2`
(B) `b2 a1`
(C) Compile error: structured bindings need C++17
(D) Compile error: cannot bind to `map` elements

Ch. 17 STL Containers

Q55 S — map vs unordered_map §17.2, p134

```
std::map<int,int>           m1 = {{3,3},{1,1},{2,2}};
std::unordered_map<int,int> m2 = {{3,3},{1,1},{2,2}};
for (auto [k,v]:m1) std::cout << k << " ";  // (A)
for (auto [k,v]:m2) std::cout << k << " ";  // (B)
```

What can be guaranteed about the output?
(A) (A) prints `1 2 3` in sorted order; (B) order is unspecified
(B) Both print `1 2 3` in sorted order
(C) Both print in insertion order
(D) (A) prints in insertion order; (B) is sorted

Q56 S — priority_queue: min or max? §17.4, p135

```
std::priority_queue<int> pq;
pq.push(3); pq.push(1); pq.push(4);
std::cout << pq.top();
```

(A) `1` (min-heap by default)
(B) `4` (max-heap by default)
(C) `3` (insertion order)
(D) Undefined (unspecified by standard)

Ch. 18 Iterators & Algorithms

Q57 S — std::remove does not erase §18.2, p138

```
std::vector<int> v = {1, 2, 3, 2, 4};
std::remove(v.begin(), v.end(), 2);
std::cout << v.size() << " " << v[0] << " " << v[3];
```

(A) Size = 3; elements are `{1, 3, 4}`
(B) Size = 5; `remove` only shuffles, does not reduce size
(C) Compile error: `remove` needs a sorted container

(D) Size = 3; but the last two elements are unspecified

Q58 S — std::sort stability §18.2, p138

(A) `std::sort` is guaranteed to be stable (equal elements keep relative order).
(B) `std::sort` is NOT guaranteed to be stable; use `std::stable_sort` for that.
(C) `std::sort` uses quicksort exclusively.
(D) `std::stable_sort` has $O(n)$ complexity.

Ch. 19 Error Handling & Exceptions

Q59 S — noexcept and std::terminate §19.5, p143

```
void f() noexcept {
    throw std::runtime_error("oops");
}
int main() {
    try { f(); }
    catch (...) { std::cout << "caught"; }
}
```

(A) Prints `caught`
(B) `std::terminate()` is called; the catch block is never reached
(C) Compile error: cannot throw in a `noexcept` function
(D) UB: behaviour depends on the compiler

Q60 S — RAII and exceptions §19.7, p144

```
struct Guard {
    ~Guard() { std::cout << "cleanup "; }
};
void risky() {
    Guard g;
    throw std::runtime_error("err");
}
int main() {
    try { risky(); } catch (...) { std::cout << "caught"; }
}
```

What is printed?
(A) `caught`
(B) `cleanup caught`
(C) `caught cleanup`
(D) Nothing (terminate is called)

Q61 M — Exception safety levels §19.6, p143

Which are correct? (Pick ONE or MORE)
☐ **A)** The **basic guarantee**: the program remains in a valid state, no resources are leaked.
☐ **B)** The **strong guarantee**: if an exception is thrown, state is rolled back as if the operation never happened.
☐ **C)** The **no-throw guarantee**: the function is marked `noexcept`.
☐ **D)** **No guarantee**: the basic guarantee applies if nothing is stated.

Part 3 Move Semantics, Templates & Modern C++

Ch. 20 Move Semantics & Value Categories

Q62 S — lvalue vs rvalue §20.1.1, p145

Which of the following are **rvalues**?

(A) `int x = 5;` — `x` is an rvalue

(B) `x + 1` and `42` are rvalues

(C) `&x` is an rvalue

(D) All named variables are rvalues

Q63 S — std::move does not move §20.1.3, p146

```
std::string s = "hello";
std::string t = std::move(s);
std::cout << s.size() << " " << t;
```

What is printed? (Assume a standard-compliant implementation)

(A) `5 hello` — `move` does nothing

(B) `0 hello` — `s` is in a valid but unspecified (likely empty) state

(C) `5` — `t` is now empty

(D) Undefined behaviour

Q64 S — When move is called §20.1.2–3, p146

```
std::string make() { return std::string("world"); }
std::string a = make();          // (1)
std::string b = std::move(a);    // (2)
```

Which operations trigger a move (not a copy)?

(A) Only (2)

(B) Only (1), because the return is a temporary

(C) Both (1) and (2) (RVO/NRVO elides even the move in (1))

(D) Neither — both are copies

Ch. 21 Templates & Compile-Time Programming

Q65 S — Template instantiation site §21.1.1, p149

```
// util.h
template <typename T>
T maxOf(T a, T b) { return a > b ? a : b; }

// main.cpp
#include "util.h"
maxOf(3, 7);
```

Where does the compiler generate the code for `maxOf<int>`?

(A) In `util.h` (header), shared by all translation units

(B) At the call site in `main.cpp`, inside that translation unit
(C) In a separate template translation unit
(D) The linker generates it after all TUs are compiled

Q66 S — if constexpr discards branches §2.4, 21.3, p19

```
template <typename T>
void print(T x) {
    if constexpr (std::is_pointer_v<T>)
        std::cout << *x;
    else
        std::cout << x;
}
print(42);   // (A)
print(&42);  // (B) -- note: &42 is ill-formed; pretend T=int*
```

For call (A) with `T=int`, what happens to the `*x` branch?
(A) It is compiled but never executed
(B) It is completely removed from compilation for `T=int`
(C) Compile error: `*42` is invalid and is still compiled
(D) The whole function fails to instantiate

Q67 S — constexpr vs consteval §21.4.1, p151

```
consteval int always_ct(int x) { return x * 2; }
int n; std::cin >> n;
int a = always_ct(5);   // (A)
int b = always_ct(n);   // (B)
```

(A) Both (A) and (B) compile
(B) (A) compiles; (B) is a compile error (`n` is not a constant expression)
(C) (B) compiles; (A) is a compile error
(D) `consteval` behaves the same as `constexpr`

Q68 S — const_cast: defined or UB? §21.5.1, p153

```
const int ci = 10;
int* p = const_cast<int*>(&ci);
*p = 99;  // (A)
std::cout << ci;
```

(A) Prints `99` — const_cast legally removes constness
(B) UB: modifying a `const` object through a cast is undefined behaviour
(C) Compile error: const_cast cannot apply to `int`
(D) Prints `10` — the compiler optimizes away the write

Q69 S — Signed integer overflow §21.6, p154

```
int x = INT_MAX;
x = x + 1;
std::cout << x;
```

(A) Prints `INT_MIN` (wraps around, well-defined)
(B) Undefined behaviour: signed overflow has no defined behaviour in C++
(C) Compile error: overflow detected at compile time

(D) Prints 0

Ch. 22 Timing, Formatting & Advanced Class Features

Q70 S — Rule of Zero §22.4, p158

A class uses only `std::vector` and `std::unique_ptr` as members. What does the Rule of Zero say?

(A) You must define all five special members (constructor, destructor, copy/move ctor/assign).

(B) You should define none of the five — the compiler-generated ones will do the right thing.

(C) You must define the destructor but nothing else.

(D) You must define the copy constructor to prevent shallow copies of the pointer.

Q71 S — RAII scope guard §22.6, p158

```
struct Guard {
    std::function<void()> fn;
    ~Guard() { fn(); }
};
void process() {
    Guard g{[] { std::cout << "done\n"; }};
    if (some_error) return;  // early exit
    // ... more work
}
```

When `some_error` is true, when does `"done"` print?

(A) It never prints on early return

(B) It prints at the `return` statement (destructor fires on scope exit)

(C) It prints after `process()` returns to its caller

(D) Only if you explicitly call `g.fn()`

Ch. 23–24 Attributes & C++20 Concepts

Q72 S — [[nodiscard]] §23.1, p161

```
[[nodiscard]] int compute() { return 42; }
compute();        // (A) --- return value discarded
int x = compute(); // (B)
```

(A) Both compile without warning

(B) (A) produces a compiler warning; (B) is fine

(C) (A) is a compile error; (B) is fine

(D) Both produce warnings

Q73 S — C++20 Concept constraint §24.1, p163

```
template <std::integral T>
T add(T a, T b) { return a + b; }

add(1, 2);       // (A)
add(1.0, 2.0);   // (B)
```

(A) Both compile
(B) (A) compiles; (B) is a compile error (double is not std::integral)
(C) (B) compiles; (A) is a compile error
(D) Concepts only produce runtime errors

Ch. 25 Multithreading & Concurrency

Q74 S — Data race §25.1–3, p165

```
int counter = 0;
void inc() { for (int i=0; i<100000; ++i) ++counter; }
std::thread t1(inc), t2(inc);
t1.join(); t2.join();
std::cout << counter;
```

(A) Always prints `200000`
(B) Undefined behaviour: unsynchronized concurrent writes
(C) Prints `100000`
(D) Compile error: `counter` is not atomic

Q75 S — std::atomic guarantee §25.3, p166

Replace `int counter = 0;` with `std::atomic<int> counter{0};` in the code above.
(A) Still UB: atomics only prevent data races, not incorrect values
(B) Guaranteed to print `200000`
(C) May print any value: memory order is unspecified
(D) Compile error: `++` is not defined on `atomic<int>`

Q76 M — mutex and lock_guard §25.2, p165

Which are correct? (Pick ONE or MORE)
☐ **A)** `std::lock_guard` is an RAII wrapper that releases the mutex in its destructor.
☐ **B)** `std::lock_guard` can be unlocked manually before destruction.
☐ **C)** `std::unique_lock` supports deferred locking and manual unlock.
☐ **D)** Calling `mutex.lock()` twice from the same thread (without unlock) deadlocks.

Ch. 26 Common Patterns & Idioms

Q77 S — Copy-and-swap idiom §26.1, p169

```
MyClass& operator=(MyClass rhs) {  // (A) pass by value
    swap(*this, rhs);
    return *this;
}
```

Why is the parameter taken **by value** in copy-and-swap?
(A) It avoids the self-assignment check and handles both copy and move assignment in one function.
(B) It is a mistake; parameters should always be const references in assignment operators.
(C) It avoids calling `swap`, which would be recursive.
(D) By-value copy is always faster than by-reference copy.

Q78 S — std::span: non-owning view §24.2, p163

```
void process(std::span<int> data) {
    for (auto& x : data) x *= 2;
}
int arr[] = {1, 2, 3, 4};
process(arr);
std::cout << arr[0];
```

(A) Prints `1` — `span` is a copy

(B) Prints `2` — span is a non-owning view; modifications affect original

(C) Compile error: cannot pass C-array to `span`

(D) UB: span has no bounds checking

Q79 M — Useful std::algorithm interview patterns §26.3, p170

Which pairs are correct? (Pick ONE or MORE)

☐ **A)** `std::accumulate(v.begin(),v.end(),0)` — sums all elements.

☐ **B)** `std::find(v.begin(),v.end(),x)` — returns iterator to first `x`, or `end()` if not found.

☐ **C)** `std::count_if` counts elements satisfying a predicate.

☐ **D)** `std::transform` sorts a range in-place using a comparator.

Detailed Answers & Explanations

Q1

Answer: C — Preprocessor → Compiler → Linker — Text substitution first (macros, includes), then each TU compiled to `.o`, then the linker stitches `.o` files.

Q2

Answer: B — Compiler needs full body at instantiation site — Templates are not code by themselves; the compiler generates concrete code per type at the call site. If the body is in a `.cpp`, it is invisible to other TUs and instantiation fails.

Q3

Answer: A, B, D — (C) `#pragma once` is a widely-supported extension, NOT part of the ISO standard.

Q4

Answer: B — pointer to Foo compiles — Forward declaration gives the compiler a name and size-promise for pointers/references. Calling methods or creating by value requires the full definition.

Q5

Answer: C — `5 100` — Inside `demo`, the local `x=5` shadows the global. `::x` explicitly accesses the global `x=100`.

Q6

Answer: A — Only (1) — (1) `constexpr int a = square(5)` — constant context forces compile-time. (2) `n` is a runtime variable; `constexpr` falls back to runtime. (3) `auto c` is mutable — also runtime fallback. Only a **constexpr** call-site variable forces compile-time.

Q7

Answer: A — Line A only — `const int f()` is a plain runtime function (`const` on a value return type is silently discarded). Using its result in `constexpr auto x` requires a compile-time call — which fails. Lines B and C just call it at runtime; those are fine.

Q8

Answer: A, B, C, D — All four are correct. `const` (member qualifier) ≠ `constexpr` (specifier): orthogonal axes. `constexpr` implies `inline` (standard mandated).

Q9

Answer: B — `name1`: string copy; `name2`: const string& — `auto` drops `const` and references by default, so `name1` is a **copy**. Writing `const auto&` preserves the reference.

Q10

Answer: A — int, unsigned, long long, double — Literal suffix rules: no suffix → `int`; `u` → `unsigned int`; `LL` → `long long`; no suffix on `3.14` → `double`.

Q11

Answer: B — int, double — `decltype(x)` is the type of `x` = `int`. `decltype(x + 3.0)` is the type of the expression `int + double` = `double`.

Q12

Answer: B — a=-3, b=-1, c=1 — C++ truncates integer division toward zero: $-7/2 = -3$. Modulo sign follows the **dividend** (left operand): $-7\%2 = -1$; $7\% - 2 = 1$.

Q13

Answer: B — a=7, b=6, c=6 — `++a`: a becomes 6, then b=6. `a++`: c gets old value (6), then a becomes 7.

Q14

Answer: A — 8 14 6 — AND = $1000_2 = 8$; OR = $1110_2 = 14$; XOR = $0110_2 = 6$.

Q15

Answer: B — 8 then 40 — Inside the function, `arr` has decayed to a pointer (8 bytes on 64-bit). Outside (global scope), `arr` is still the full array: $10 \times 4 = 40$ bytes.

Q16

Answer: C — two three other — No `break` after `case 2`. Execution falls through into `case 3`, which also falls through into `default`.

Q17

Answer: C — Compile error: jump over initialization — A `goto` (which `switch` implements internally) from `case 2:` past the initialization of `x` in `case 1:` is ill-formed. Fix: wrap in `{}`.

Q18

Answer: B — {2, 4, 6} — only loop B modifies — Loop A iterates by **value copy**; doubling `x` does not touch the vector. Loop B uses a reference and modifies in-place. After A: `{1,2,3}`; after B: `{2,4,6}`.

Q19

Answer: B — invalid: non-trailing default — Default arguments must trail from the right. `void f(int a=1, int b, int c=3)` has a gap (`b` has no default between two defaulted parameters) — ill-formed.

Q20

Answer: B — 10 20, no effect — `swap(int a, int b)` takes by value. Inside the function `a` and `b` are copies; the originals `x` and `y` are unchanged.

Q21

Answer: B — Compile error: redefinition — Overloading is resolved on parameter types only. Two functions differing only in return type are a **redefinition**, not an overload.

Q22

Answer: B — Multiple definitions allowed (no ODR violation) — `inline` on a function primarily controls the linker, not the optimizer. It allows the same definition to appear in multiple TUs.

Q23

Answer: B — Mixing printf/scanf with cin/cout becomes unsafe — After disabling sync, the two buffer systems are independent. Interleaving them produces garbled output.

Q24

Answer: A, B, D — (C) After a `failbit`, all subsequent extractions silently fail until `cin.clear()` is called.

Q25

Answer: B — cerr unbuffered, clog buffered — `cerr` flushes on every write (fd 2, unbuffered). `clog` buffers writes on fd 2 for efficiency.

Q26

Answer: B — Undefined behaviour: dangling view — `string_view` is a non-owning pointer+length into the `string`. When `s` goes out of scope, the storage is freed; `sv` becomes a dangling reference.

Q27

Answer: B — const char*, string, string_view — Undecorated string literals are `const char*`. The `s` suffix creates `std::string`. The `sv` suffix creates `std::string_view`.

Q28

Answer: C — 1 2 3 — The `static` local `n` is initialized to 0 once. Each call increments it: first call returns 1, second 2, third 3.

Q29

Answer: A, C, D — (B) `static` locals live in the data/BSS segment, not the heap. (A) Initialized exactly once. (C) C++11 mandates thread-safe static local init (magic statics). (D) `thread_local` — per-thread storage.

Q30

Answer: B — 20 10 (swap succeeds) — Both `a` and `b` are lvalue references to the callers' `x` and `y`. The swap modifies the originals directly.

Q31

Answer: C — Prints 8 (lifetime extended) — Binding a temporary to a `const` lvalue reference extends the temporary's lifetime to match the reference's scope. This is a well-defined C++ rule.

Q32

Answer: A — 8 8 8 — `sizeof(double) = 8`. `sizeof(double*) = 8` on 64-bit. `sizeof(*ptr) = sizeof(double) = 8`.

Q33

Answer: C — UB (likely segfault) — Dereferencing a null pointer is undefined behaviour. There is no meaningful value to print.

Q34

Answer: B — Linker error — Global `const` variables default to **internal linkage** in C++. The symbol `LIMIT` is invisible outside `file_a.cpp`; the `extern` declaration in `file_b.cpp` causes a linker error. Fix: use `inline const int LIMIT = 100;` or `extern const int LIMIT;` with explicit `extern const int LIMIT = 100;` definition.

Q35

Answer: A, B — (A) `f('a')` exactly matches the deleted `f(char)` — compile error ("use of deleted function"). (B) `f(5LL)`: `long long` → `int` and `long long` → `char` are both viable with equal rank — ambiguity error. (C) `f(3.14)`: `double` → `int` is a standard conversion; it resolves to `f(int)` (compiles).

Q36

Answer: B — result=3.5, wrong=3.0 — (A) Casts 7 to `double` *before* division: `7.0/2 = 3.5`. (B) Evaluates `7/2=3` (integer division) *first*, then casts 3 to `double`: `3.0`.

Q37

Answer: C — -1 — `find(7)` returns `nullopt`. `value_or(-1)` returns the fallback `-1` when the optional is empty.

Q38

Answer: B — (A) compiles, (B) is a compile error — Unscoped `enum` implicitly converts to `int`. Scoped `enum class` does NOT implicitly convert; you must write `static_cast<int>(Dir::Left)`.

Q39

Answer: B — 99 (shallow copy) — The compiler-generated copy constructor copies all members byte-for-byte, including the raw pointer `data`. Both `a.data` and `b.data` point to the same `int`. Modifying through `b.data` changes what `a.data` also sees.

Q40

Answer: B — (A) compile error, (B) compiles — A `const` object can only call `const`-qualified member functions. `deposit` is not `const`, so `w.deposit(50)` fails. `get()` is `const`, so it is fine.

Q41

Answer: C — 0 copies with mandatory RVO — C++17 mandates copy elision (RVO) in this case. The `string` object is constructed directly in the storage of `s`. Zero copies, zero moves.

Q42

Answer: B — emplace_back: constructs in-place — `push_back({1,2})` creates a temporary `pair` then moves it into the vector. `emplace_back(3,4)` forwards the arguments and constructs the `pair` directly inside the vector's buffer.

Q43

Answer: B — 10 99 — `f` captures `x` by value (copy). `mutable` allows modifying the copy inside `f`, but the outer `x` stays 10. After `f()`, outer `x` is still 10. `g` captures by reference; `g()` sets outer `x` to 99.

Q44

Answer: B — 3 (captured at creation) — `[=]` captures *copies* of `a` and `b` at the point the lambda is **created** (a=1, b=2). Subsequent changes to the outer variables do not affect the captured copies.

Q45

Answer: B — UB: iterator invalidated — `vector::erase` invalidates *all* iterators at and after the erased element. Incrementing `it` after erasure is UB. Correct pattern: `it = v.erase(it)` (skip `++it` in the branch).

Q46

Answer: A, B, C — (D) False — Inheritance is the *tightest* coupling (not loosest). Composition > Aggregation > Association > (no relation); Inheritance sits alongside/above Composition in coupling strength.

Q47

Answer: B — Prints Animal (static binding) — Without `virtual`, the call `p->speak()` is resolved at compile time based on the pointer type (`Animal*`). Dog's override is never reached.

Q48

Answer: B — Prints Dog — With `virtual` + `override`, the vtable is consulted at runtime. `p` holds a `Dog` object, so `Dog::speak()` is called.

Q49

Answer: B — Compile error — `override` forces the compiler to verify that the function matches a virtual in the base. `process(double)` does not match `process(int)` — compile error. This is exactly the protection `override` provides.

Q50

Answer: B — Prints ~Base only (UB: Derived members not cleaned up) — Without a `virtual` destructor, `delete p` calls `Base:: Base` only. `Derived:: Derived` is never called — any Derived-specific resources are leaked. This is undefined behaviour per the standard.

Q51

Answer: A — Both are compile errors — An abstract class (with at least one pure virtual) cannot be instantiated at all — neither on the stack nor on the heap.

Q52

Answer: A, B, C, D — Without virtual inheritance, `D` has two `A` sub-objects (one via `B`, one via `C`). `d.x` is ambiguous. Fix: virtual inheritance, which uses a vbptr (virtual base pointer) to share a single `A` instance.

Q53

Answer: B — throws std::bad_variant_access — The variant currently holds a `double`. Requesting `std::get<int>` on a variant holding a different type throws `std::bad_variant_access` at runtime.

Q54

Answer: A — a1 b2 — `std::map` is a sorted BST; iteration is in ascending key order. Structured bindings (`auto& [key, val]`) require C++17 and work on map pairs.

Q55

Answer: A — map sorted, unordered_map unspecified — `std::map` (red-black tree) always iterates in sorted key order. `std::unordered_map` iterates in bucket order, which is implementation-defined.

Q56

Answer: B — 4 (max-heap) — `std::priority_queue<int>` is a max-heap by default. `top()` returns the largest element.

Q57

Answer: B — size=5, remove only shuffles — `std::remove` moves matching elements to the end and returns a new-end iterator, but does **not** resize the container. Call `v.erase(std::remove(...), v.end())` (erase-remove idiom) to actually shrink.

Q58

Answer: B — std::sort is not stable — `std::sort` is $O(n \log n)$ but not stable. Use `std::stable_sort` (also $O(n \log n)$, but with a larger constant) for stability.

Q59

Answer: B — std::terminate() is called — Throwing from a `noexcept` function calls `std::terminate()` immediately. The exception does not propagate; the catch block is unreachable.

Q60

Answer: B — cleanup caught — Stack unwinding during exception propagation calls destructors of all local variables in scope. `Guard:: Guard()` fires first (printing `"cleanup "`), then the catch block runs (printing `"caught"`).

Q61

Answer: A, B, C — (D) False — the "no guarantee" level is below basic; without stating any guarantee, no guarantee is implied. The basic guarantee is the minimum accepted level in well-written code.

Q62

Answer: B — x+1 and 42 are rvalues — Named variables like `x` are lvalues (they have an identifiable memory location). Temporaries and literals (`42`, `x+1`) are rvalues. `&x` yields a pointer (lvalue).

Q63

Answer: B — 0 hello — `std::move(s)` casts `s` to an rvalue reference, enabling the move constructor of `string`. After the move, `s` is in a valid but unspecified state (typically empty). `t` owns the data.

Q64

Answer: C — RVO/NRVO elides even the move in (1) — In (1), mandatory RVO (C++17) elides the temporary entirely — no move or copy. In (2), `std::move` enables move construction of `b` from `a`.

Q65

Answer: B — at call site in main.cpp — The compiler instantiates a template function at the call site within the translation unit. The template body must be visible there (hence the header requirement).

Q66

Answer: B — completely removed from compilation for T=int — `if constexpr` discards the false branch before instantiation. `*x` is **never compiled** for `T=int`, which is exactly the purpose.

Q67

Answer: B — (A) compiles, (B) is a compile error — `consteval` functions *must* be evaluated at compile time. `n` is a runtime variable — passing it to `always_ct` is a hard error.

Q68

Answer: B — UB: modifying a truly const object — `ci` is a truly `const` object. Using `const_cast` to write through a pointer to it is undefined behaviour. The compiler may optimize based on the assumption that `ci` never changes, so the output could be 10 or 99 or anything.

Q69

Answer: B — Undefined behaviour — Signed integer overflow is UB in C++. The compiler may assume it never happens and optimize accordingly. The result is *not* guaranteed to wrap to `INT_MIN`.

Q70

Answer: B — Define none; compiler-generated ones are correct — When resource-managing members (RAII types like `unique_ptr`, `vector`) handle all lifetime concerns, the compiler-generated special members work correctly. The Rule of Zero says: if you don't need to manage resources manually, don't define any of the five.

Q71

Answer: B — prints at the return statement (scope exit) — The destructor of `g` is called when the scope of `process()` exits, whether by `return` or by falling off the end. RAII guarantees cleanup on any exit path.

Q72

Answer: B — (A) warning, (B) fine — `[[nodiscard]]` causes a **compiler warning** (not error) when the return value is discarded. It does not prevent compilation.

Q73

Answer: B — (A) compiles, (B) compile error — `std::integral` is satisfied by integer types only. `double` is not integral — compile error with a readable constraint failure message.

Q74

Answer: B — UB: data race — Concurrent unsynchronized writes to `counter` from multiple threads is a data race — undefined behaviour in C++.

Q75

Answer: B — Guaranteed 200000 — `std::atomic<int>` makes `++counter` an indivisible operation. All increments are serialized; the final value is always exactly 200000.

Q76

Answer: A, C, D — (B) False — `lock_guard` cannot be unlocked manually (use `unique_lock` for that). (A) RAII: locked in constructor, unlocked in destructor. (C) `unique_lock` is flexible. (D) `mutex::lock()` from the same thread is UB (for non-recursive mutexes).

Q77

Answer: A — Avoids self-assignment check; handles both copy and move — The by-value parameter invokes the copy or move constructor depending on what is passed. Inside the function body, `rhs` is a local copy/moved-into object; swapping is always safe, even on self-assignment.

Q78

Answer: B — Prints 2 (modifications affect original) — `std::span` is a non-owning view (pointer + length). Modifying elements through the span modifies the underlying array. `arr[0]` becomes `2`.

Q79

Answer: A, B, C — (D) False — `std::transform` applies a function to elements and stores results (possibly in-place); sorting is done by `std::sort`.

Quick Answer Key

Q	Topic	Answer
1	Compilation order	C
2	Template placement	B
3	pragma once vs guard	A, B, D
4	Forward declaration	B
5	Scope resolution ::	C
6	constexpr: when runs	A
7	const return type	A
8	const vs constexpr	A,B,C,D
9	auto drops const&	B
10	Literal suffixes	A
11	decltype result	B
12	Integer division/modulo	B
13	Pre vs post increment	B
14	Bitwise AND/OR/XOR	A
15	sizeof pointer in fn	B
16	Switch fallthrough	C
17	Switch variable init	C
18	Range-for copy vs ref	B
19	Default args: invalid?	B
20	swap by value	B
21	Overload on return type	B
22	inline meaning	B
23	FastIO trade-off	B
24	Stream state bits	A,B,D
25	cerr vs clog	B
26	string_view dangling	B
27	String literal types	B
28	Static local counter	C
29	Static properties	A,C,D
30	Swap with reference	B
31	Const ref + temporary	C
32	sizeof double vs ptr	A
33	nullptr deref	C
34	Global const linkage	B
35	Implicit conversion/delete	A,B
36	static_cast order matters	B
37	optional value_or	C
38	enum class conversion	B
39	Shallow copy	B
40	const member function	B
41	RVO copies	C
42	emplace_back efficiency	B
43	Lambda capture by val/ref	B
44	Lambda default capture	B
45	Erase inside loop	B
46	Object relations	A,B,C
47	Without virtual: binding	B
48	With virtual: dispatch	B
49	override catches typo	B
50	Missing virtual destructor	B
51	Abstract class instance	A
52	Diamond problem	A,B,C,D
53	variant bad access	B
54	Structured bindings map	A
55	map vs unordered iteration	A
56	priority_queue default	B
57	std::remove behaviour	B
58	std::sort stability	B
59	noexcept + throw	B
60	RAII + exception	B
61	Exception safety	A,B,C
62	lvalue vs rvalue	B
63	std::move effect	B
64	When move is called	C
65	Template instantiation	B
66	if constexpr discards	B